Vehicle Rental Management System

1. Title Page:

INSTITUTE OF ENGINEERING AND MANAGEMENT

COURSE NAME: ADVANCED PROGRAMMING (OOP)

PROJECT TITLE: Vehicle Rental Management System

STUDENT NAME: ANKAN KANRAR

ROLL NUMBER: 52

ENROLLMENT NUMBER: 12024052017054

INSTRUCTOR NAME: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

SEMESTER: SPRING 2026

2. Problem Statement:

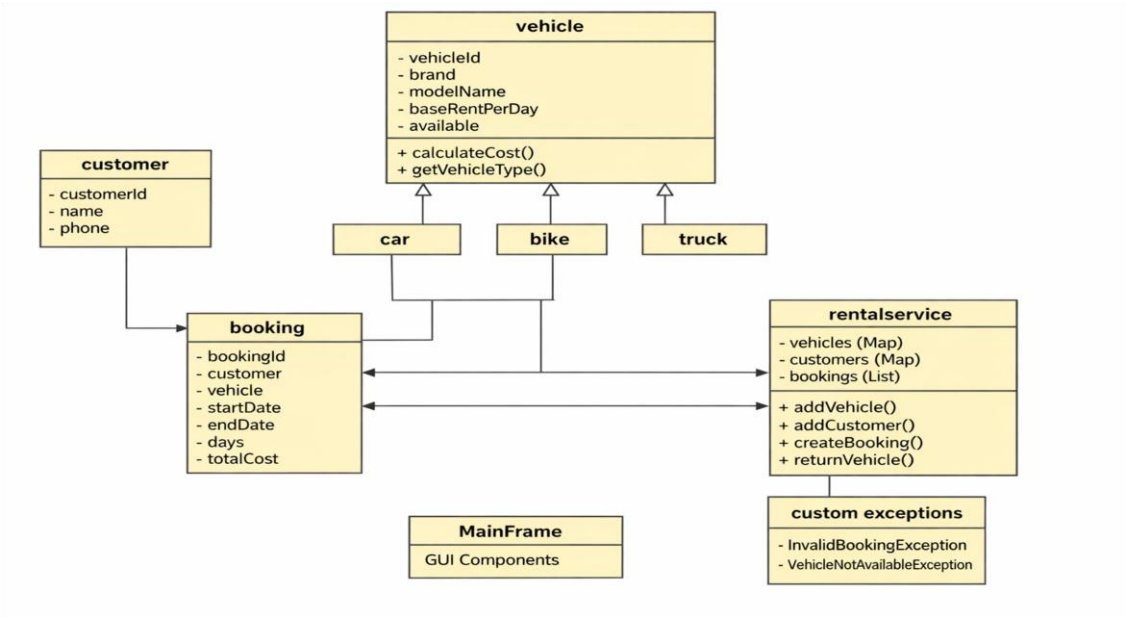
Managing vehicle rentals manually can be difficult because it involves tracking vehicles, customers, booking dates, availability, and rental charges. In a paper-based or unstructured system, mistakes can occur during booking, vehicle assignment, and cost calculation. This project addresses those issues by creating a desktop-based Vehicle Rental Management System using Java and Swing.

The system allows users to add vehicles, register customers, create bookings, and view records in a structured interface. It also checks vehicle availability and calculates rental cost based on vehicle type and duration. The project demonstrates core Object-Oriented Programming concepts such as encapsulation, inheritance, polymorphism, abstraction, exception handling, and the use of collections. Since the application is designed using a layered structure, the code is easier to maintain, extend, and understand.

This system is useful for small rental shops and also serves as a practical OOP project that shows how real-world entities can be modeled in Java.

3. System Design:

3.1 UML Class Diagram:



3.2 Description of Major Classes:

Vehicle:
Base class storing common attributes like ID, brand, model, rent, and availability. Defines abstract methods for cost calculation.

Car, Bike, Truck:
Derived classes that extend `Vehicle` and implement their own rental cost logic (polymorphism).

Customer :
Stores customer details such as ID, name, and phone number.

Booking:
This class stores booking information including booking ID, customer, vehicle, start date, end date, number of days, and total cost.

RentalService:
This class acts as the service layer. It stores vehicles, customers, and bookings using HashMap and ArrayList, and handles booking creation, vehicle availability checks, and record management.

MainFrame:
This is the GUI class built using Java Swing. It contains forms, buttons, tabs, and tables for user interaction.

Custom Exceptions:
These are used to handle invalid booking duration and unavailable vehicle conditions in a controlled way.

3.3 Package Structure

The project is organized into four main packages:

- model/ — contains entity classes such as Vehicle, Customer, Booking, Car, Bike, and Truck
- service/ — contains the business logic class RentalService
- gui/ — contains the Swing GUI classes MainFrame and VehicleRentalApp
- exception/ — contains custom exception classes

This structure separates data, logic, interface, and error handling, making the project easier to understand and maintain.

3.4 Interaction Flow

1. The application starts from VehicleRentalApp.
2. MainFrame opens the GUI.
3. The user adds vehicles and customers through forms.
4. When booking is created, the system checks customer and vehicle availability.
5. If validation passes, booking is stored in memory.
6. Records are displayed in tables inside the GUI.
7. Exceptions are shown to the user through dialog boxes if invalid input is entered.

4. OOP Concepts Demonstrated

4.1 Encapsulation

Where it is used:
In classes like Vehicle, Customer, and Booking.

Why it is used:
Encapsulation protects data by keeping fields private and allowing access through methods.

Snippet:

```
private String vehicleId;

private String brand;
```

```
private String modelName;


public String getVehicleId() {
    return vehicleId;
}
```

Explanation:
The variables are hidden from direct access, which prevents accidental modification. Getter methods are used to read the values safely. This improves data security and code control.

4.2 Abstraction

Where it is used:
In the Vehicle class.

Why it is used:
Abstraction hides implementation details and focuses on essential behavior common to all vehicles.

Snippet:

```
public abstract class Vehicle {
    public abstract double calculateRentalCost(int days);
    public abstract String getVehicleType();
}
```

Explanation:
The Vehicle class defines common structure but does not fully implement cost calculation. Each subclass provides its own logic. This simplifies the design and allows flexible extension.

4.3 Inheritance

Where it is used:
Car, Bike, and Truck inherit from Vehicle.

Why it is used:
Inheritance reduces redundancy by sharing common features from a parent class.

Snippet:

```
public class Car extends Vehicle {
    public Car(String vehicleId, String brand, String modelName, double baseRentPerDay) {
        super(vehicleId, brand, modelName, baseRentPerDay);
    }
}
```

Explanation:
The Car class gets all common vehicle properties from Vehicle. Only specific behavior needs to be added in the child class. This avoids repeated code.

4.4 Polymorphism

Where it is used:
In the overridden calculateRentalCost() method of each vehicle type.

Why it is used:

Polymorphism allows different vehicle types to behave differently using the same method name.

Snippet:

```
@Override
public double calculateRentalCost(int days) {
    return baseRentPerDay * days * 0.8;
}
```

Explanation:
Each vehicle type can compute rental cost differently. This makes the system more realistic because different vehicles may have different pricing rules.

4.5 Collections Framework

Where it is used:
In RentalService.

Why it is used:
Collections are used to store and manage multiple records efficiently.

Snippet:

```
private Map<String, Vehicle> vehicles = new HashMap<>();
private Map<String, Customer> customers = new HashMap<>();
private List<Booking> bookings = new ArrayList<>();
```

Explanation:
HashMap is used for quick lookup of vehicles and customers by ID. ArrayList is used to maintain booking records in order. This makes data handling efficient and organized.

4.6 Exception Handling

Where it is used:
In booking creation and validation logic.

Why it is used:
Exception handling prevents program crashes and gives meaningful error messages.

Snippet:

```
if (!vehicle.isAvailable()) {
    throw new VehicleNotAvailableException("Vehicle is already booked.");
}
```

Explanation:
If a vehicle is already booked, the system throws a custom exception instead of failing silently. This improves reliability and user feedback.

5. Implementation Highlights

5.1 Vehicle Cost Calculation

The vehicle classes implement cost calculation based on type. This allows different pricing logic for different vehicle categories.

Snippet:

```
@Override
public double calculateRentalCost(int days) {
```

```
    return baseRentPerDay * days * 1.25;
}
```

Justification:
This design demonstrates polymorphism and allows flexible pricing rules without changing the main service logic.

5.2 Booking Creation Logic

The service layer validates dates and creates bookings only when the vehicle is available.

Snippet:

```
if (startDate == null || endDate == null || !endDate.isAfter(startDate)) {
    throw new InvalidBookingDurationException("End date must be after start date.");
}
```

Justification:
This prevents invalid booking periods and improves input validation.

5.3 Vehicle Availability Update

When a booking is made, the selected vehicle is marked unavailable.

Snippet:

```
vehicle.setAvailable(false);
```

Justification:
This ensures the same vehicle cannot be booked by multiple users at the same time.

5.4 GUI-Based User Interaction

The Swing interface allows users to work with the system through forms and tables.

Snippet:

```
JTabbedPane tabs = new JTabbedPane();
tabs.addTab("Vehicles", createVehiclePanel());
tabs.addTab("Customers", createCustomerPanel());
tabs.addTab("Bookings", createBookingPanel());
```

Justification:
Tabbed navigation keeps the interface organized and beginner-friendly. It improves usability and makes the application easy to demonstrate.

6. Testing & Error Handling

6.1 Test Cases Considered

Test Case	Input	Expected Result
Add valid vehicle	Vehicle ID, brand, model, rent	Vehicle added successfully
Add valid customer	Customer ID, name, phone	Customer added successfully

Test Case	Input	Expected Result
Create valid booking	Existing customer and available vehicle	Booking created
Invalid date range	End date before start date	Error message displayed
Booking unavailable vehicle	Vehicle already booked	Custom exception shown
Invalid number input	Text entered in numeric field	Parsing error handled

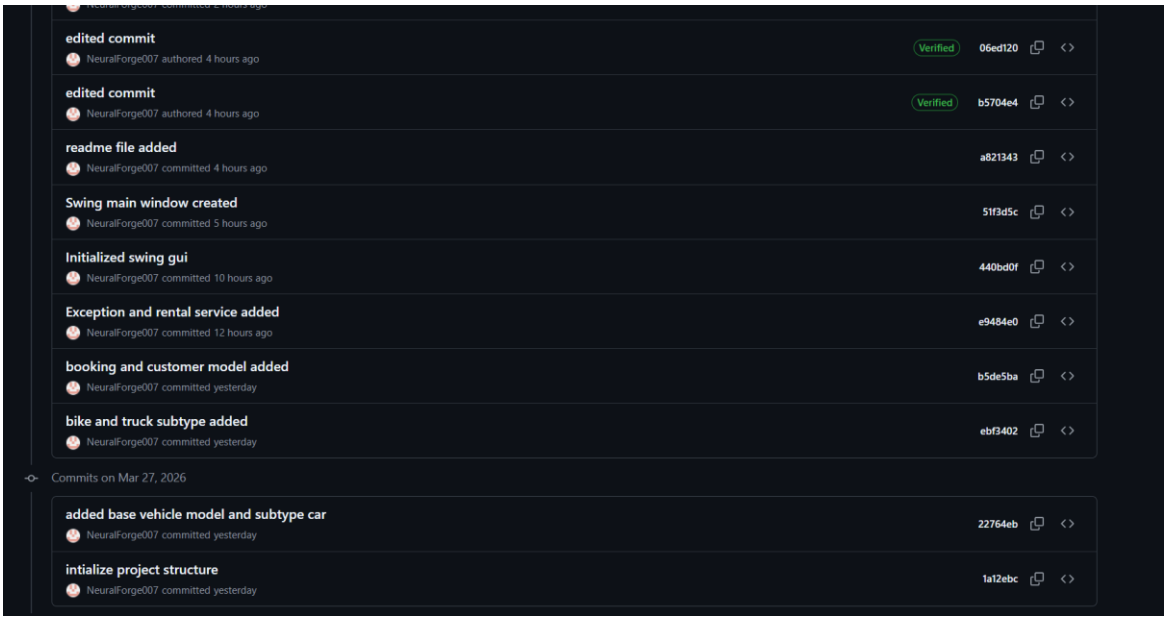
6.2 Edge Cases Handled

- Empty input fields
- Invalid date format
- End date earlier than start date
- Booking with non-existing customer or vehicle
- Vehicle already booked
- Invalid numeric rental price

6.3 Failure Scenarios

If a user enters wrong data, the system does not crash. Instead, it shows an error dialog. For example, if booking duration is invalid or a vehicle is unavailable, custom exceptions are used to notify the user. This improves robustness and user experience.

7. Git Workflow



Development Progression

The project was developed in multiple stages instead of a single upload. The development started with repository setup and folder creation. After that, model classes were added, followed by the service layer, exception handling, GUI development, and documentation updates. Each stage was committed separately with descriptive commit messages.

This workflow shows proper version control discipline and demonstrates incremental progress. The commit history helps verify that the project was developed over time and not submitted as a single bulk upload.

8. Conclusion & Future Scope

Conclusion

The Vehicle Rental Management System successfully demonstrates how Object-Oriented Programming concepts can be applied to solve a real-world problem. The project models vehicles, customers, and bookings using classes and objects. It also uses inheritance, abstraction, polymorphism, collections, and exception handling in a practical way. The Swing-based interface makes the system easier to use and understand.

Future Scope

The project can be improved further by adding:

- Database integration using MySQL
- Login and authentication system
- Search and filter options
- Booking cancellation feature
- Better UI design
- Persistent data storage
- Receipt or bill generation

COPY OF INITIAL PROJECT PROPOSAL

TOPIC :- Vehicle Rental Management System using Object-Oriented Programming

Problem Statement :

Many vehicle rental services rely on manual systems or poorly structured software, which causes issues such as booking conflicts, incorrect availability tracking, and inefficient customer management. A digital system is needed to manage vehicles, customers, reservations, and payments efficiently. The proposed Vehicle Rental Management System will allow users to browse available vehicles, make bookings, and manage reservations, while administrators manage inventory and rental records. Using Object-Oriented Programming concepts like abstraction, inheritance, and polymorphism, the system will be modular, scalable, and capable of handling booking errors.

Target Users

- Customers who want to rent vehicle,Rental company staff / administrators,Vehicle fleet managers

Core Features

- User registration and login system
- Browse and search available vehicles
- Vehicle booking and reservation management
- Rental cost calculation based on duration
- Vehicle availability tracking.

OOP Concepts to be Used

Abstraction: Abstract classes such as Vehicle will define common properties like vehicle ID, rental price, and availability while hiding internal implementation details.

Inheritance: Different vehicle types like Car, Bike, and Truck will inherit from the base Vehicle class.

Polymorphism: Methods like calculateRentalCost() will behave differently for different vehicle types depending on pricing models.

Exception Handling

Custom exceptions will handle situations such as:

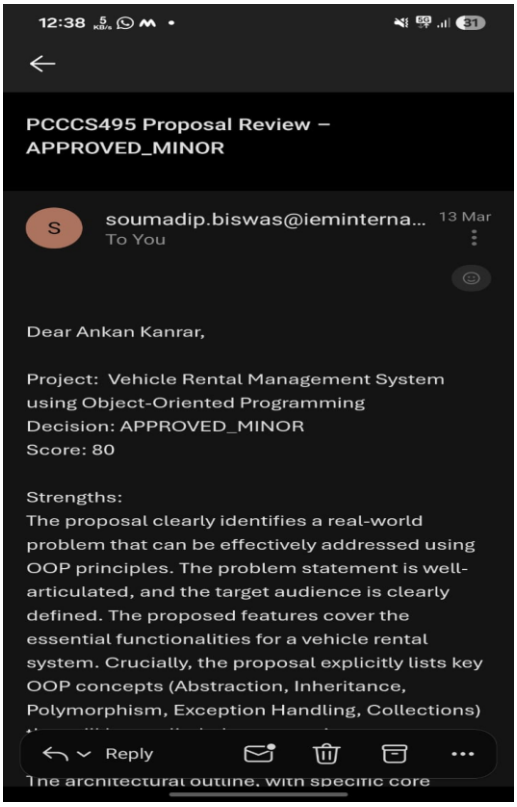
- Vehicle not available,Invalid booking duration,Customer not found

Collections / Threads: Collections such as lists or maps will be used to store vehicles, customers, and booking records dynamically.

Proposed Architecture Description

The system will follow a **modular object-oriented architecture** consisting of multiple interacting classes. The core modules will include Vehicle, Customer, Booking, Payment, and RentalService. The RentalService class will act as the central controller managing bookings, vehicle availability, and customer interactions. Data will be stored using collections to manage vehicles and bookings efficiently. The system will implement proper exception handling to prevent invalid operations and ensure reliable system behavior.

COPY OF REVIEW MAIL RECEIVED-APPROVED



PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminterna... 13 Mar
To You

Dear Ankan Kanrar,

Project: Vehicle Rental Management System using Object-Oriented Programming
Decision: APPROVED_MINOR
Score: 80

Strengths:
The proposal clearly identifies a real-world problem that can be effectively addressed using OOP principles. The problem statement is well-articulated, and the target audience is clearly defined. The proposed features cover the essential functionalities for a vehicle rental system. Crucially, the proposal explicitly lists key OOP concepts (Abstraction, Inheritance, Polymorphism, Exception Handling, Collections)

Reply

The architectural outline, with specific core